

Foundation of Blockchain Technology (IS501)
Pre-Reading Material – Lecture 20

Control Flow in Solidity

Estimated Reading Time: 15–20 minutes

Introduction

In Solidity, smart contracts often need to **make decisions** and **repeat operations**. For example, a contract may need to check whether a customer qualifies for a discount or process multiple items in an order.

Flow control determines the order in which statements are executed.

The main flow control structures in Solidity are:

Flow Control

Purpose

if–else	Makes decisions based on conditions
while	Repeats while a condition is true
do–while	Executes at least once and then checks a condition
for	Repeats operations using a counter or fixed range

if–else Statement

The **if–else statement** is used for decision-making.

- The **if** block executes when a condition is true.
- The **else** block executes when the condition is false.

Example

In an online donut shop:



If a customer purchases 12 or more donuts, give a dozen discount; otherwise, do not give the discount.

Key Points

- Used for conditional decisions.
- Only the appropriate branch is executed.
- Curly braces { } are used to group multiple statements.
- Variables should be properly initialized before use.

while Loop

A **while loop** repeatedly executes a block of statements **as long as a condition remains true**.

Example

A contract may process donuts one by one until the required quantity is reached.

Important Features

- The condition is checked **before** executing the loop body.
- The loop may execute **zero or more times**.
- A **break** statement can be used to exit the loop early.
- The loop should have a clear stopping condition.

Important: Loops consume blockchain gas. An unnecessarily long or infinite loop can cause transaction execution to fail when the available gas is exhausted.

do-while Loop

A **do-while loop** is similar to a while loop, but the condition is checked **after** the loop body.

Therefore, the loop body executes **at least once**.

Difference

Loop Condition Checked Minimum Executions

while Before the loop body 0

do-while After the loop body 1

Example

A donut-order process may need to execute once before checking whether additional processing is required.

for Loop

The **for loop** is commonly used when the number of iterations can be controlled using a counter.

It generally contains three parts:

1. **Initialization** – starts the counter.
2. **Condition** – determines whether the loop continues.
3. **Increment/Decrement** – changes the counter.

Example

A contract can use a for loop to process donuts from 1 up to the purchased quantity.

Advantages

- Compact and structured.
- Suitable for counter-based repetition.
- Keeps initialization, condition, and increment together.
- Useful when the iteration range is known.

break Statement

The **break** statement immediately terminates a loop.



For example, if a contract finds that the required quantity has been reached, it can use break to stop further processing.

Key Point

break helps avoid unnecessary iterations and can reduce unnecessary computation.

7. Flow Control Comparison

Structure	Main Purpose	Condition Check
if-else	Decision-making	Before selecting a branch
while	Repeated execution	Before each iteration
do-while	Repeated execution	After each iteration
for	Counter-based repetition	Before each iteration

8. Important Solidity Considerations

Gas Usage

Every operation executed on the blockchain consumes computational resources and may consume **gas**. Therefore, loops should be designed carefully.

Initialization

Local variables should be initialized appropriately before they are used.

Loop Termination

Every loop should have a clear condition that eventually stops execution.

Scope

Local variables declared inside functions generally have a limited scope and do not interfere with similarly named variables in other functions.

Readability

Use meaningful variable names, proper indentation, comments, and curly braces to make smart contracts easier to understand and maintain.

9. Practical Example

Consider an online donut shop smart contract:

- **if-else:** Check whether the customer qualifies for a bulk discount.
- **while:** Continue processing until the required quantity is reached.
- **do-while:** Perform an operation at least once before checking the condition.
- **for:** Process a fixed or counter-based number of items.
- **break:** Stop processing when the required condition is satisfied.

10. Key Takeaways

- **if-else** is used for decision-making.
- **while** repeats while a condition is true and may execute zero times.
- **do-while** executes at least once before checking the condition.
- **for** is useful for counter-based or fixed-range repetition.
- **break** can terminate a loop early.
- Loops should be **short, finite, and gas-efficient**.
- Flow control is essential for implementing business rules and automated logic in Solidity smart contracts.

